

```
1, dot: 1, sign: 1, .: 1, It: 1, is:
1, important: 1, to: 1, detect: 1,
sentences: 1, in: 1, nlp: 1, .: 1})
```

POS tagging

Parts of speech (POS) tagging helps us in breaking down a sentence and understanding what role each word in the sentence plays. There are eight parts of speech in the English language, i.e., noun, pronoun, adjective, verb, adverb, preposition, conjunction and interjection.

In POS tagging, we apply a tag to each word in a sentence that defines what part of speech that word represents in the context of a given sentence. In spaCy, we will make use of two properties — *tag*, which gives the fine-grained part of speech, and *pos*, which gives the coarse-grained part of speech. The *spacy.explain()* method provides a description of a given tag.

```
#POS tagging
for word in read_text:

    print(word, word.tag_, word.pos_,
          spacy.explain(word.pos_))
```

The following is the output of the POS tagging of the sample text. In order to preserve page space, I am providing only a portion of the output.

```
This DT DET determiner
is VBZ AUX auxiliary
an DT DET determiner
example NN NOUN noun
of IN ADP adposition
a DT DET determiner
passage NN NOUN noun
```

Visualising POS tagging using displaCy

spaCy comes with a built-in visualiser called *displaCy*, using which we can apply and visualise parts of speech (POS) tagging and named entity recognition (NER). To visualise POS tagging for a sample text, run the following code:

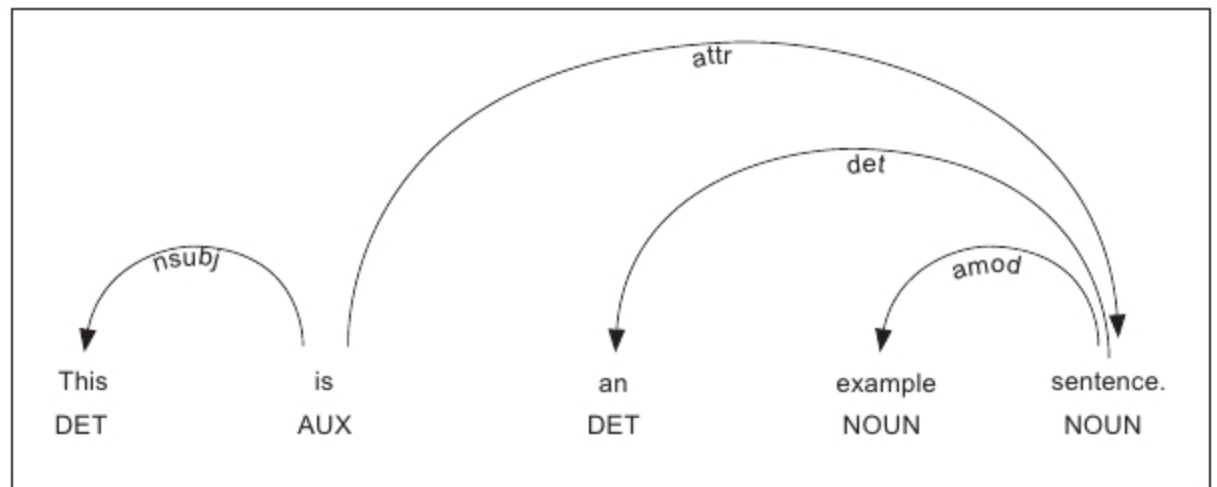


Figure 1: POS tagging visualisation using displaCy



Figure 2: NER visualisation using displaCy

```
#using displacy for POS tag
visualisation
from spacy import displacy
sample_text = "This is an example
sentence."
read_text = myspacy(sample_text)
displacy.serve(read_text, style="dep")
```

Figure 1 shows the output of the code, which you can view by using your browser on the localhost link provided by *displaCy*.

Visualising NER using displaCy

Named entity recognition (NER) is the process of identifying named entities in a piece of text and tagging them with a pre-defined category, such as a name of a person, location, organisation, percentage, time, etc. spaCy has the property *ents*, which we can use to apply NER on text. We will also use *displaCy* to visualise the NER output.

```
#using displacy for NER visualisation
sample_text = "Every year, Hyderabad
hosts the biggest exhibition in India.
It has fun rides and food stalls. At
```

```
least a million people visit it every
year."
read_text = myspacy(sample_text)
displacy.serve(read_text, style="ent")
```

Figure 2 shows the output of the NER visualisation.

In this article, we have discussed the basic functionalities most commonly used for NLP using spaCy and its built-in visualiser *displaCy*. There are many more advanced features in this library that are absolutely worth exploring and mastering if you want a solid foundation in NLP. However, those features are beyond the scope of this article. Hence, this is a good stopping point. I have shared a few links as reference to help you get a deeper understanding of the spaCy library. I recommend you go through them. **END** 🐧

References

- [1] <https://spacy.io/models>
- [2] <https://spacy.io/usage>

By: Mir H.S. Quadri

The author is a scholar and researcher in the fields of artificial intelligence and machine learning. He shares a deep love for Web development and has worked on multiple projects using a wide array of frameworks. He is also a FOSS enthusiast and actively contributes to several open source projects. You can find his works on codelatte.site.